Databricks Design 1

CSV-Driven Predictive Optimization Control

Implementation-ready design specification for Claude Code / Claude-assisted development

Purpose
Build a reusable Databricks notebook utility that enables or disables Predictive Optimization (PO) at catalog, schema, or table scope from a CSV stored in a Unity Catalog Volume. A wrapper notebook collects configuration, creates and starts a serverless-compatible Databricks Job, and reports the run. A worker notebook validates rows, executes DDL independently, and writes an XLSX result report to a Volume.

Design principles

- Fail independently per input row; one DDL failure must not stop later rows.
- No audit table and no persistent database objects created by the utility.
- Use validated structured inputs; never accept raw SQL from the CSV.
- Distinguish row-level failures from bootstrap, infrastructure, and report-writing failures.
- Keep wrapper orchestration separate from worker execution.

Version: 1.0 **Prepared:** 15 October 2026 **Status:** Design baseline; verify workspace-specific API and PO syntax before production.

1. Scope and requirements

In scope

• CSV-driven ENABLE / DISABLE for catalog, schema, and table objects in Unity Catalog.

• Notebook widgets for Volume root, CSV path, XLSX report path, worker notebook path, job name prefix, wait option, and polling timeout.

• Wrapper creates a one-task Databricks Job and submits a run using serverless-compatible notebook task configuration.

• Worker validates the complete input structure before starting DDL, then processes valid rows independently.

• Excel workbook includes a summary sheet and row-level details.

• Errors are documented in the workbook when possible and in job output for failures that prevent report creation.

Out of scope

• Audit tables, metrics tables, or other persistent database objects.

• Accepting arbitrary SQL or shell commands from CSV.

• Automatically retrying failed DDL statements.

• Guaranteeing successful XLSX output when the target Volume or file writer is unavailable.

Success criteria

A run produces a job/run identifier, attempts every non-conflicting valid CSV row, records each outcome, and writes the report. Row-level failures do not fail the worker if the report is written. Bootstrap failures and report-generation failures must remain visible as task failures.

2. Architecture

Component	Responsibilities
Wrapper notebook	Defines widgets; validates configuration and paths; creates the Job; starts the run; optionally polls; displays job ID, run ID, state, and report path.
Databricks Job	One notebook task targeting the worker notebook; configured for serverless compute where supported by workspace, region, permissions, and task settings.
Worker notebook	Reads task parameters; validates CSV; constructs safe SQL; executes one statement per row; collects result records in memory; writes XLSX; prints summary.
Unity Catalog Volume	Stores the input CSV and output XLSX. No audit table or persistent control table is created.

Flow

```
Wrapper widgets
   -> validate paths and configuration
   -> create one-time Job (worker notebook task)
   -> submit run and retain job_id / run_id
   -> optionally poll run state
Worker on serverless-compatible compute
   -> read CSV -> validate rows -> generate SQL
   -> execute each valid row independently
   -> write Summary + Details sheets to XLSX
   -> print totals and report path
```

Important: do not rely on interactive wrapper state inside the worker. Pass all worker inputs as task parameters.

3. CSV contract and validation

Input columns (exact required header)

```
object_type,catalog,schema,table,action
CATALOG,finance,,,DISABLE
SCHEMA,sales,staging,,ENABLE
TABLE,sales,curated,orders,DISABLE
TABLE,sales,curated,customers,ENABLE
```

object_type	Required fields	Allowed action	Command shape
CATALOG	catalog	ENABLE / DISABLE	ALTER CATALOG `` ENABLE|DISABLE PREDICTIVE OPTIMIZATION
SCHEMA	catalog, schema	ENABLE / DISABLE	ALTER SCHEMA ``.`` ENABLE|DISABLE PREDICTIVE OPTIMIZATION
TABLE	catalog, schema, table	ENABLE / DISABLE	ALTER TABLE ``.``.`` ENABLE|DISABLE PREDICTIVE OPTIMIZATION

Scope syntax and support must be confirmed against the target Databricks workspace's current SQL reference before implementation. Do not silently assume a syntax form is valid for all object types.

Validation rules

• Require the exact header set; reject missing required columns. Extra columns should either be rejected (recommended) or explicitly ignored and documented; do not guess.

• Trim whitespace; normalize object_type and action to uppercase.

• Required identifiers must be non-empty; fields irrelevant to the selected scope should be blank. Reject inconsistent rows.

• Accept only ENABLE or DISABLE for version 1.0. Do not implement INHERIT unless separately requested and verified.

• Build SQL only from the enumerated object_type and action values.

• Quote each identifier independently with backticks; escape an embedded backtick by doubling it. Never treat identifier values as SQL fragments.

• Track original CSV row number (header is row 1) for report traceability.

• Detect duplicate targets using normalized scope and identifier tuple.

• For identical duplicate requests, execute once and mark later duplicates SKIPPED_DUPLICATE. For conflicting actions on the same target, reject all rows for that target as SKIPPED_CONFLICT; do not use last-row-wins.

• Validate paths and file extensions before DDL. Do not execute anything if the CSV cannot be read or its structure is invalid.

Object eligibility and privileges

Predictive Optimization applies to eligible Unity Catalog managed tables; do not assume external tables, foreign catalogs, or all catalog types support the same operation. The job identity must have the privileges required for each target. Parent-level settings may not override explicitly configured child settings. Report command success as command success—not proof that every child object's effective behavior changed.

4. Wrapper notebook widgets

Widget	Example/default	Validation
volume_root	/Volumes/admin/ops/po_control	Must be a valid Unity Catalog Volume root; use as allowed-path boundary.
csv_path	/Volumes/admin/ops/po_control/po_operations.csv	Must be within volume_root; readable; distinct from report_path.
report_path	/Volumes/admin/ops/po_control/po_report.xlsx	Must be within volume_root; .xlsx extension; must not resolve to csv_path.
worker_notebook_path	/Shared/po_control/po_worker	Non-empty workspace notebook path; caller can access it.
job_name_prefix	po-control	Sanitize for job name; append a unique run suffix if needed.
wait_for_completion	true	Boolean string; if false, return after run submission.
max_wait_minutes	60	Positive integer; polling timeout only, not a job execution timeout.

Use dbutils.widgets for the wrapper inputs. Pass csv_path and report_path to the worker as notebook task base_parameters. The worker should declare matching widgets and read them with dbutils.widgets.get().

5. Job creation and serverless execution

The wrapper should create a one-task Databricks Job through the supported Jobs API or Databricks SDK, then submit a run. Use the currently supported serverless Jobs notebook-task configuration for the target workspace. Do not copy a generic API payload blindly: validate the current API schema and required serverless fields in the deployed workspace.

Job configuration requirements

• One task, with a stable task_key such as execute_po_operations.

• Notebook task path set to worker_notebook_path; source setting matches whether the notebook is in Workspace or Git-backed source.

• Task parameters include csv_path, report_path, and any other required non-secret settings.

• Configure serverless using the supported Jobs API mechanism for the workspace; do not specify a classic cluster unless deliberately switching design.

• Set max_concurrent_runs to 1 for the generated job where practical; prevent accidental overlap against the same output file.

• Use an approved job identity. Do not embed tokens, passwords, or secrets in widget values, job parameters, or report.

• Capture job_id and run_id immediately and show them to the user.

• If polling times out, report state as still running or unknown based on the last API response. Do not mark it failed or cancel it automatically.

Lifecycle recommendation

Prefer a reusable job that the wrapper updates or reuses when feasible, rather than creating unlimited permanent jobs. If the requirement explicitly mandates a newly created job per invocation, name it with a timestamp/run suffix and define a cleanup policy. Do not delete a job while its run is active.

Permissions and runtime checks

• Job creator has permission to create/run jobs and access the worker notebook.

• Run identity has the necessary UC privileges on target objects and read/write privileges on the Volume.

• Serverless is available and permitted in the workspace/region, and the selected task supports it.

• Pandas and an Excel writer (for example, openpyxl) are available to the worker. Configure dependencies in a supported job environment if needed.

6. Worker execution algorithm

```
1. Read csv_path and report_path task parameters.
2. Validate both paths and ensure report_path != csv_path.
3. Initialize run metadata and in-memory result records.
4. Read CSV while preserving original row numbers and malformed-row information.
5. Validate file-level structure. If unusable, fail before executing any DDL.
6. Validate rows; group by normalized target; mark invalid/conflicting/duplicate rows.
7. For each remaining valid, unique target:
     a. Generate SQL from the allowlisted scope/action.
     b. Execute with spark.sql(sql_text) in a row-level try/except.
     c. Record SUCCESS or EXECUTION_FAILED and error details.
     d. Continue to next target regardless of row-level failure.
8. Build Summary and Details sheets in memory.
9. Write XLSX to a temporary path, then publish final report where supported.
10. If report writing fails, log the full error and fail the task.
11. Otherwise print totals/report path and exit successfully, even if rows failed.
```

Do not wrap the entire row loop in one try/except. Use a separate exception boundary for each SQL operation. Do not swallow exceptions from initialization, CSV loading, or final report generation.

7. Report specification

Output workbook: XLSX file at report_path. It contains two sheets.

Sheet	Required fields
Summary	run_id if available; job_id if passed; input CSV path; report path; start/end timestamps with timezone; total input rows; valid unique targets attempted; successes; validation failures; execution failures; duplicates skipped; conflicts skipped; report status; overall outcome.
Details	source_row_number; object_type; catalog; schema; table; fully_qualified_target; requested_action; generated_sql; status; start/end timestamp or execution timestamp; error_class; error_message; duplicate/conflict reference where applicable.

Status values

Status	Meaning
SUCCESS	spark.sql returned without raising an exception.
VALIDATION_FAILED	Row did not pass validation.
EXECUTION_FAILED	SQL execution raised an exception.
SKIPPED_DUPLICATE	Same normalized target and same action was already handled.
SKIPPED_CONFLICT	Target had conflicting actions; none of its conflicting rows executed.

Report implementation details

• Use pandas plus openpyxl or another supported Excel writer.

• Create the report directory only if the Volume filesystem semantics and permissions support it; otherwise fail clearly.

• Write to a unique temporary file and publish to final path only if the runtime supports reliable rename/replace semantics. Validate actual behavior on the chosen Volume/FUSE environment.

• Define overwrite policy. Recommended: require a unique report filename per run, or explicitly allow overwrite after validation. Never overwrite input CSV.

• Preserve complete error messages in the report. If any message is truncated for spreadsheet limits, log the full exception in task output and state that it was truncated.

• Escape or neutralize spreadsheet formula-leading cell values in user-controlled text fields when writing XLSX, so CSV identifiers/errors cannot become formulas.

• Do not write secrets or access tokens into the report.

Overall status

If the XLSX is successfully written, the worker task may complete successfully even when some rows have VALIDATION_FAILED or EXECUTION_FAILED statuses. The summary must make partial failure obvious. File/bootstrap errors and report-generation errors should fail the task so the platform surfaces them.

8. Failure-handling matrix

Failure	Required behavior	Task outcome
Invalid row	Record error; continue other rows.	Success if report writes.
DDL fails / permission denied	Record SQL error; continue.	Success if report writes.
Conflicting duplicate	Mark all conflicting rows; execute none for target.	Success if report writes.
CSV missing/unreadable or header invalid	No DDL; print diagnostic; fail clearly.	Failed.
Worker cannot start / job API fails	Wrapper displays API error and any known IDs.	No successful run claim.
Polling timeout	Return last known state and run_id; do not cancel.	Worker state may be running/unknown.
Report path unwritable / XLSX writer fails	Log exception to task output; fail task.	Failed.
Serverless unavailable	Surface job/task configuration error; do not fall back silently to classic compute.	Failed.

9. Security and correctness requirements

• Allowlist object types and actions; do not accept SQL text in the CSV.

• Escape each SQL identifier safely and quote catalog/schema/table separately.

• Never execute any DDL until file-level CSV structure and path validation have passed.

• Do not automatically retry DDL after an exception or timeout; outcome may be ambiguous. Require explicit rerun after checking state.

• No secrets in widgets, task parameters, logs, or reports.

• Enforce Volume-root path boundary for input and output. Reject path traversal or alternate paths outside the configured root.

• Avoid logging sensitive row contents beyond object identifiers and execution diagnostics needed for operations.

• No audit tables, control tables, or extra persistent DB objects.

• No concurrency against the same report path; use a unique output path per run or a lock-free single-run policy.

10. Acceptance tests

Test case	Expected outcome
Valid catalog/schema/table ENABLE and DISABLE	Correct scope-specific SQL executes; row marked SUCCESS when command returns normally.
Invalid action or missing required identifier	VALIDATION_FAILED; no SQL generated/executed for that row; other valid rows continue.
Bad identifier containing backticks or SQL-like text	Safely quoted as identifier or rejected by validation; never executed as SQL fragment.
Conflicting actions for same normalized target	All conflicting rows marked SKIPPED_CONFLICT; no command executed for that target.
Identical duplicate target/action	One execution only; remaining rows SKIPPED_DUPLICATE.
Permission error on one object	EXECUTION_FAILED; later rows still attempted.
Empty CSV with valid header	Zero commands; valid summary/details workbook generated.
Malformed header or missing CSV	No DDL; worker fails clearly.
Report path is same as CSV or outside root	Wrapper/worker rejects before DDL.
Report output unavailable	Failure visible in job output; task fails rather than claiming complete report.
Wrapper polling times out	run_id shown; no automatic cancellation; final state not falsely inferred.
Parent and child PO settings differ	Report command outcomes; independently verify effective settings and inheritance behavior.

11. Implementation checklist for Claude

Use the following as the coding task. Implement in the existing Databricks workspace/repository context only after inspecting the project's conventions and the current Jobs API schema.

```
Implement the design in this PDF as two Databricks notebooks:
1) po_wrapper
2) po_worker

Before coding:
- Read the repository README, existing notebook conventions, dependency/environment setup,
  and any existing job utility helpers.
- Verify current Databricks SQL syntax for PO at catalog/schema/table scope.
- Verify current Jobs API/SDK fields for serverless notebook tasks in the target workspace.
- Do not guess unsupported API fields. Isolate API-specific payload construction in one helper.

Requirements:
- Wrapper widgets: volume_root, csv_path, report_path, worker_notebook_path,
```

```
  job_name_prefix, wait_for_completion, max_wait_minutes.
- Validate configuration and enforce Volume-root path boundary.
- Create and submit a single-task serverless-compatible job, pass worker parameters,
  and report job_id/run_id/state.
- Worker reads CSV with exact schema and original row numbers; validate all file-level
  structure before executing DDL.
- Only ENABLE/DISABLE in version 1.0. Generate SQL from allowlisted values, quote
  identifiers safely, detect duplicates/conflicts, and continue after each row-level error.
- Generate XLSX with Summary and Details sheets, including all invalid/failed rows.
- No audit table or other persistent DB objects.
- Row-level failures do not fail the worker if the report is written. Bootstrap and report
  failures must be surfaced as task failures.
- Do not auto-retry SQL statements. Do not silently fall back from serverless to classic.
- Add focused tests for SQL generation, quoting, validation, duplicate policy, status counts,
  and error isolation. Mock Databricks API/Spark calls where live integration is unavailable.
- Include setup/deployment instructions and a sample CSV.
- Do not claim production readiness until workspace-specific syntax, permissions, serverless
  eligibility, Volume I/O behavior, and end-to-end tests are verified.
```

12. Pre-production sign-off

Area	Sign-off evidence
SQL correctness	Official syntax verified for all scopes; tested on non-production UC objects.
Authorization	Run identity can execute required ALTER commands and read/write Volume files.
Serverless	Job runs using supported serverless configuration in target workspace; no silent fallback.
Failure isolation	A failing row does not prevent later rows; report captures row errors.
Report integrity	XLSX opens correctly, counts reconcile, and output path/overwrite behavior are verified.
Operational controls	Unique output path or overwrite policy; job lifecycle and polling behavior tested.

Design status: ready to use as a coding specification, subject to validation against the target Databricks workspace and its current SQL / Jobs API documentation.